

Error suppression and mitigation techniques

Course: Quantum Computing Lab (ID: 5841)

Faculty: Prof. Prabha Mandayam

TAs: Bhavesh Gupta and Gayathree M. Vinod

Error Suppression & Mitigation

- to improve result quality when scaling up to larger workloads

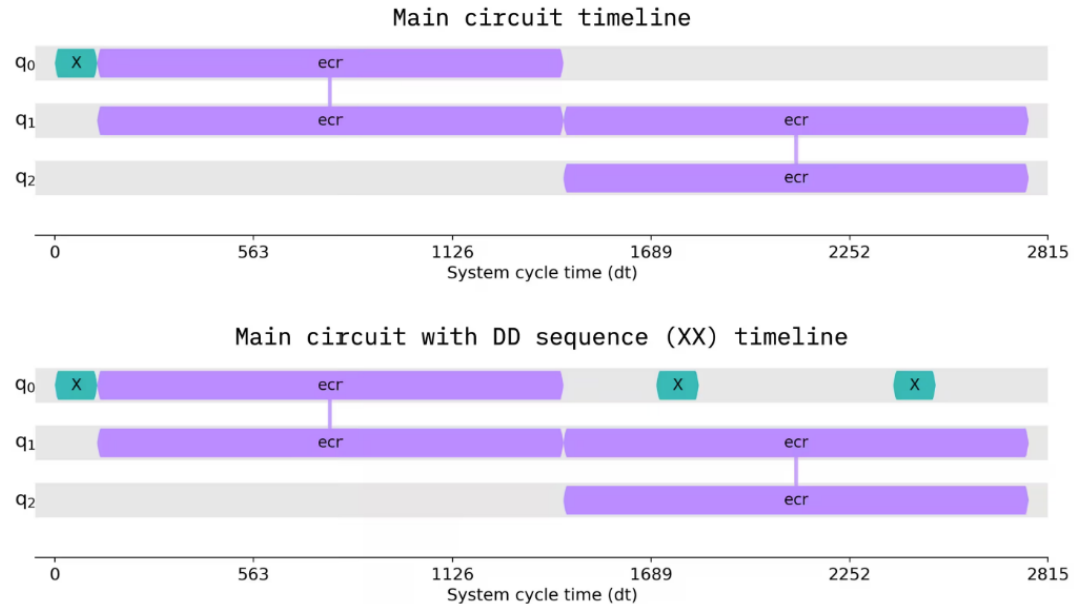
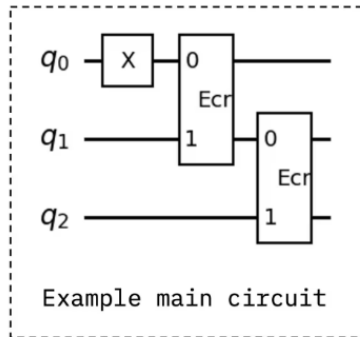
Error suppression:

- use knowledge about the undesirable effects to introduce customization that can **anticipate** and **avoid** the potential impacts of those effects
- often consist of altering or adding control signals to ensure that the quantum processor returns the desired results
- typically results in quantum **pre-processing overhead**
 - important to achieve a balance between perfecting your results and ensuring that your job completes in a reasonable amount of time
- **e.g. dynamical decoupling, Pauli Twirling**

Dynamical Decoupling:

- Quantum circuits execution on IBM® hardware → as sequences of microwave pulses scheduled and run at precise time intervals
- **unwanted interactions** between qubits → **coherent errors** on **idling qubits**
- **inserting pulse sequences on idling qubits** to approximately cancel out the effect of these errors
- each inserted pulse sequence → an **identity operation**
- the physical presence of the pulses has the effect of suppressing errors
- mainly useful for circuits containing gaps in which some qubits sit idle without any operations acting on them
- If the operations in the circuit are very densely packed → might not improve performance → could even worsen performance due to imperfections in the pulses themselves

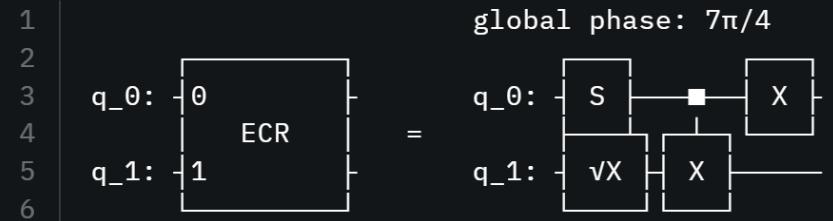
Example - Dynamical decoupling with an XX pulse sequence:



(ECR gate:

- echoed cross-resonance gate
- maximally entangling
- equivalent to a CNOT up to single-qubit pre-rotations
- the echoing procedure mitigates some unwanted terms (terms other than ZX) to cancel in an experiment
- implements $\frac{1}{\sqrt{2}} (IX - XY)$

Circuit symbol:



Matrix representation:

$$ECR_{q_0, q_1} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 & 0 & i \\ 1 & 0 & -i & 0 \\ 0 & i & 0 & 1 \\ -i & 0 & 1 & 0 \end{pmatrix}$$

Dynamical decoupling for Estimator:

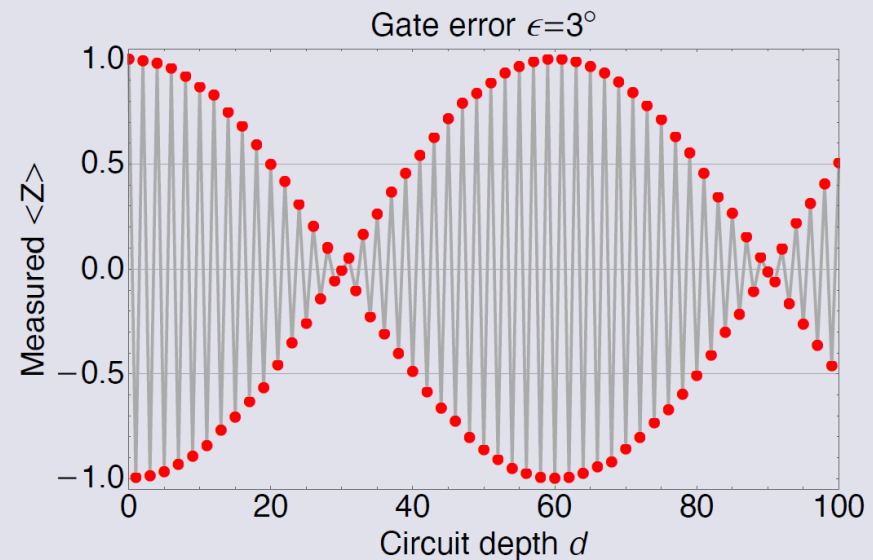
- can be enabled by setting enable to True in the 'dynamical decoupling options'
- sequence_type option → to pick from several different pulse sequences
- (default sequence type: 'XX')

```
estimator = Estimator(mode=backend)
estimator.options.dynamical_decoupling.enable = True
estimator.options.dynamical_decoupling.sequence_type = "XpXm"
```



Coherent errors: brief summary

- are ubiquitous
- can be described by unitary operations
- do not lose quantum information
- data: can create oscillations in the data
- data: do not yield exponential decays
- have a *quadratic* impact on algorithmic accuracy (worst-case error)



Zlatko Mineev, IBM Quantum (37)



Coherent error is bad, quadratically so

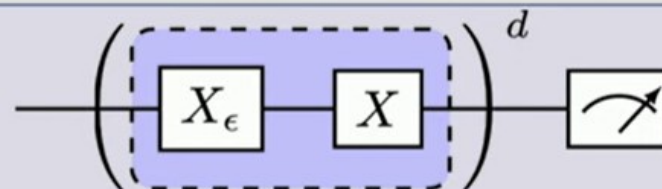
Coherent errors have a quadratic impact on algorithmic accuracy (worst-case error)

$$\langle \tilde{\psi}_f | Z | \tilde{\psi}_f \rangle - \langle \psi_f | Z | \psi_f \rangle$$

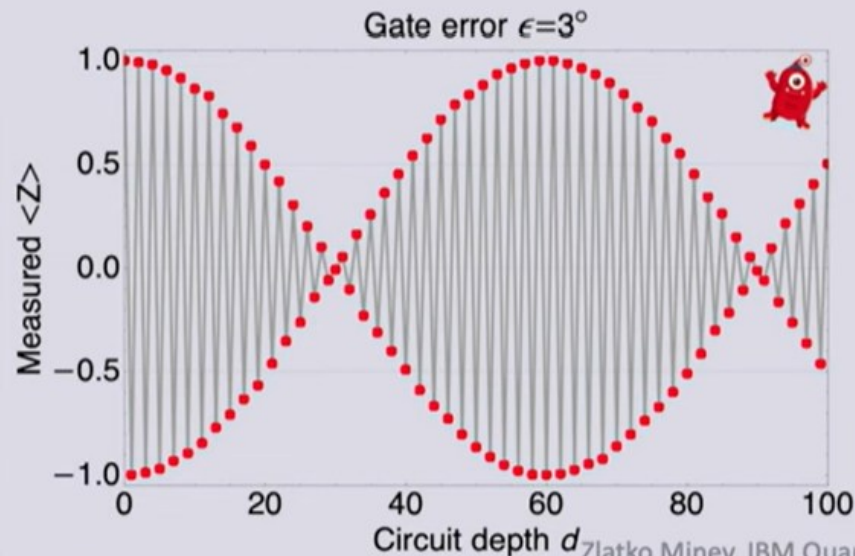
$$\cos(x) = 1 - \frac{1}{2}x^2 + \mathcal{O}(x^3)$$

$\sim \underbrace{d^2 \epsilon^2}$

Noisy



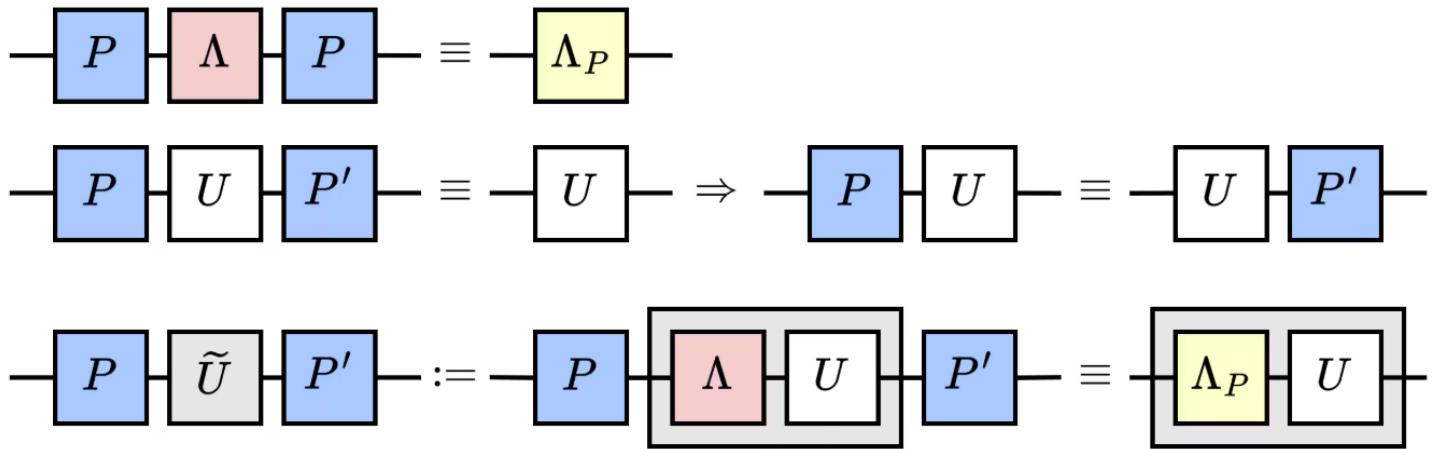
$$\langle \tilde{\psi}_f | Z | \tilde{\psi}_f \rangle = \cos(d\pi + d\epsilon)$$



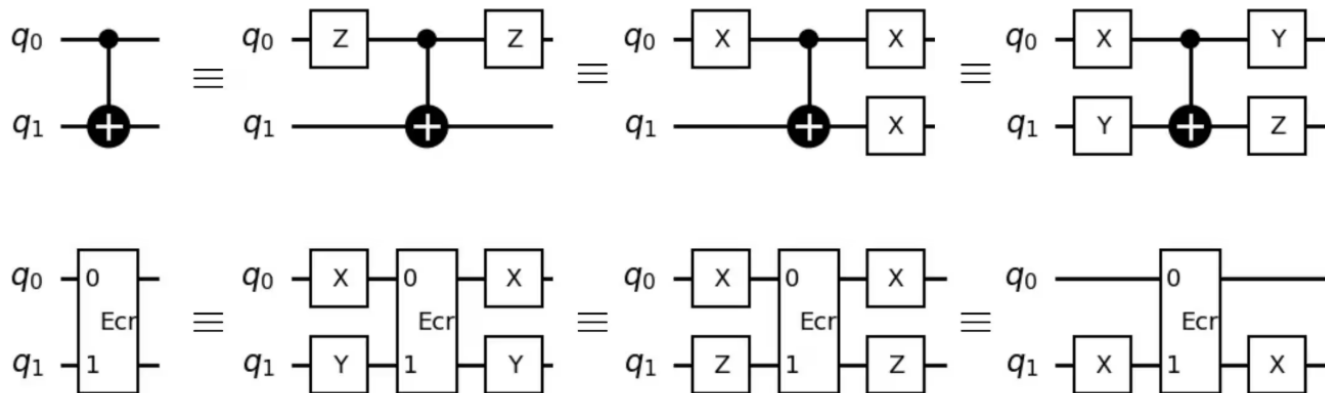
Zlatko Mineev, IBM Quantum (36)

Pauli Twirling:

- Twirling, also known as randomized compiling
- arbitrary noise channels → **noise channels with more specific structure**
- a special kind of twirling that uses **Pauli operations**
- **transforming** any quantum channel into a **Pauli channel**
- Performed alone, it can **mitigate coherent noise** because **coherent noise** tends to accumulate **quadratically** with the number of operations, whereas **Pauli noise** accumulates **linearly**
- often combined with other error mitigation techniques that work better with Pauli noise than with arbitrary noise
- implemented by sandwiching a chosen set of gates with randomly chosen single-qubit Pauli gates in such a way that the ideal effect of the gate remains the same
- Result - a single circuit is replaced with a random ensemble of circuits, all with the same ideal effect
- When sampling the circuit, samples are drawn from multiple random instances, rather than just a single one



- Since **most of the errors** in current quantum hardware come from **two-qubit gates**, this technique is often applied exclusively to (native) two-qubit gates.
- Some Pauli twirls for the CNOT and ECR gates: *(Every circuit within a row has the same ideal effect.)*



Pauli Twirling for Estimator:

- can be enabled by setting `enable_gates` to `True` in the 'twirling options'
- Other option:
 - `num_randomizations`: The number of circuit instances to draw from the ensemble of twirled circuits
 - `shots_per_randomization`: The number of shots to sample from each circuit instance

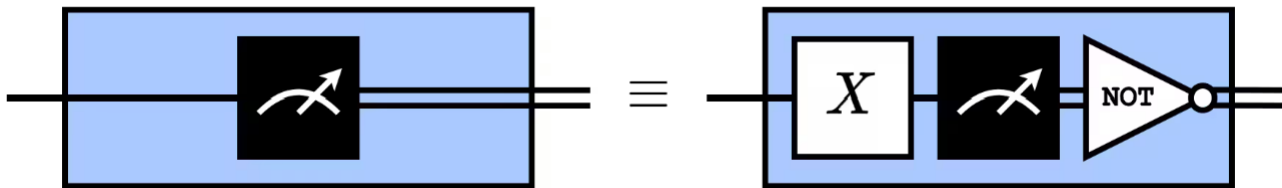
```
estimator = Estimator(mode=backend)
estimator.options.twirling.enable_gates = True
estimator.options.twirling.num_randomizations = 32
estimator.options.twirling.shots_per_randomization = 100
```

Error mitigation:

- allow users to mitigate circuit errors by **modeling the device noise at the time of execution**
- typically results in quantum **pre-processing overhead** related to **model training** and **classical post-processing overhead** to mitigate errors in the **raw results** by using the generated model
- e.g. for the Estimator: TREX, ZNE, PEC, PEA
- e.g. for the Sampler: mthree

Twirled Readout Error Extinction (TREX):

- mitigates the effect of measurement errors for the estimation of **Pauli observable expectation values**
- based on the notion of twirled measurements → accomplished by randomly substituting measurement gates by a sequence of (1) a Pauli X gate, (2) a measurement, and (3) classical bit flip (equivalent to a plain measurement in the absence of noise)



- in the presence of readout error → has the effect of diagonalizing the readout-error transfer matrix → making it easy to invert
- estimating the readout-error transfer matrix **requires executing additional calibration circuits** → introduces a **small overhead**

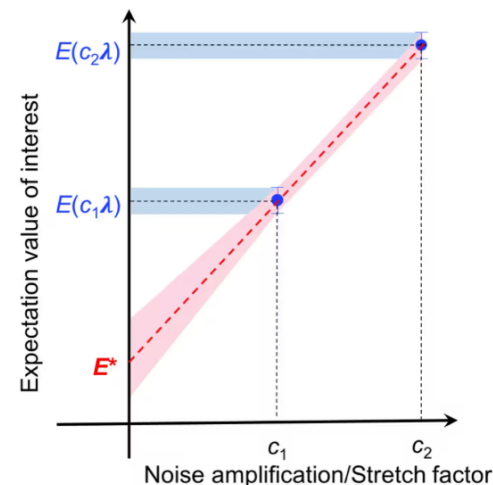
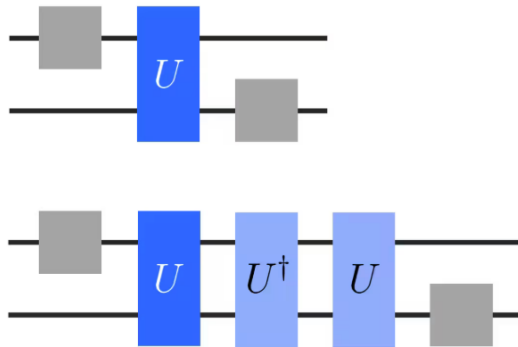
TREX for Estimator:

- can be enabled by setting `measure_mitigation` to `True` in the 'Qiskit Runtime resilience options'
- (Other option (similar to the Twirling options):
 - `num_randomizations`: The number of circuit instances to draw from the ensemble of twirled circuits
 - `shots_per_randomization`: The number of shots to sample from each circuit instance)

```
estimator = Estimator(mode=backend)
estimator.options.resilience.measure_mitigation = True
estimator.options.resilience.measure_noise_learning.num_randomizations = 32
estimator.options.resilience.measure_noise_learning.shots_per_randomization = 100
```

Zero-Noise Extrapolation (ZNE):

- for **mitigating errors** in estimating **expectation values of observables**
- While it often improves results, it is **not guaranteed to produce an unbiased result**
- Consists of **two stages**:
 - 1. **Noise amplification**: The original quantum circuit is **executed multiple times at different noise rates**
 - 2. **Extrapolation**: The ideal result is estimated by **extrapolating the noisy expectation value results to the zero-noise limit**
- both steps can be implemented in many different ways
- Qiskit Runtime noise amplification implementation: "digital gate folding" (two-qubit gates are replaced with equivalent sequences of the gate and its inverse),
 - e.g. $U \rightarrow UU^\dagger U \rightarrow$ noise amplification factor of 3
- **extrapolation choice**: **linear fit/exponential fit**



ZNE for Estimator:

- can be enabled by setting `zne_mitigation` to `True` in the 'Qiskit Runtime resilience options' for Estimator
- Other options:
 - `noise_factors`: The noise factors to use for noise amplification
 - `extrapolator`: The functional form to use for the extrapolation

```
estimator = Estimator(mode=backend)
estimator.options.resilience.zne_mitigation = True
estimator.options.resilience.zne.noise_factors = (1, 3, 5)
estimator.options.resilience.zne.extrapolator = "exponential"
```

- **one main challenge**: to **accurately amplify the noise** affecting the target circuit
gate folding – easy way to perform this amplification,
but potentially inaccurate; might lead to incorrect results
- **Probabilistic error amplification**: provides a **more accurate approach to error amplification** through **noise learning**

Probabilistic Error Amplification (PEA):

- a more sophisticated technique
- performs **preliminary experiments to reconstruct the noise**, uses this information to perform an **accurate amplification**
- starts **by learning the twirled noise model** of each layer of entangling gates in the circuit **before they are run**
- After the learning phase, the circuits are **executed at each noise factor**, where **every entangling layer of the circuits is amplified by probabilistically injecting single-qubit noise proportional to the corresponding learned noise model**
- Consists of three stages:
 - 1. **Learning**: The twirled noise model of each layer of entangling gates in the circuit is learned.
 - 2. **Noise amplification**: The original quantum circuit is executed multiple times at different noise factors
 - 3. **Extrapolation**: The ideal result is estimated by extrapolating the noisy expectation value results to the zero-noise limit
- **often the best choice for utility-scale experiments**

PEA for Estimator:

- also need to enable ZNE by setting `resilience.zne_mitigation = True` (since PEA is a ZNE noise amplification technique)
- Other `resilience.zne` options can additionally be used to set extrapolators, amplification levels, and so on
- PEA requires a noise model → automatically generated when using primitives

```
estimator = Estimator(mode=backend)
estimator.options.resilience.zne_mitigation = True
estimator.options.resilience.zne.amplifier = "pea"
```

Probabilistic Error Cancellation (PEC):

- a technique for mitigating errors in estimating expectation values of observables
- Unlike ZNE, it **returns an unbiased estimate** of the **expectation value**
- it generally incurs a **greater overhead**
- In PEC, the **effect of an ideal target circuit** is expressed as a **linear combination of noisy circuits** that are actually implementable in practice:

$$\mathcal{O}_{ideal} = \sum_i \eta_i \mathcal{O}_{noisy,i}$$

- The output of the ideal circuit can then be reproduced by executing different noisy circuit instances drawn from a random ensemble defined by the linear combination.
- If the coefficients η_i form a probability distribution, they can be used directly as the probabilities of the ensemble.

In practice, some of the coefficients are negative, so they form a **quasi-probability distribution** instead. They can still be used to define a random ensemble, but there is a **sampling overhead** related to the negativity of the quasi-probability distribution, which is characterized by the quantity

$$\gamma = \sum_i |\eta_i| \geq 1.$$

- The **sampling overhead** is a **multiplicative factor on the number of shots** required to estimate an expectation value to a given precision, compared to the number of shots that would be needed from the ideal circuit. It **scales quadratically with γ** , which in turn **scales exponentially with the depth of the circuit**.

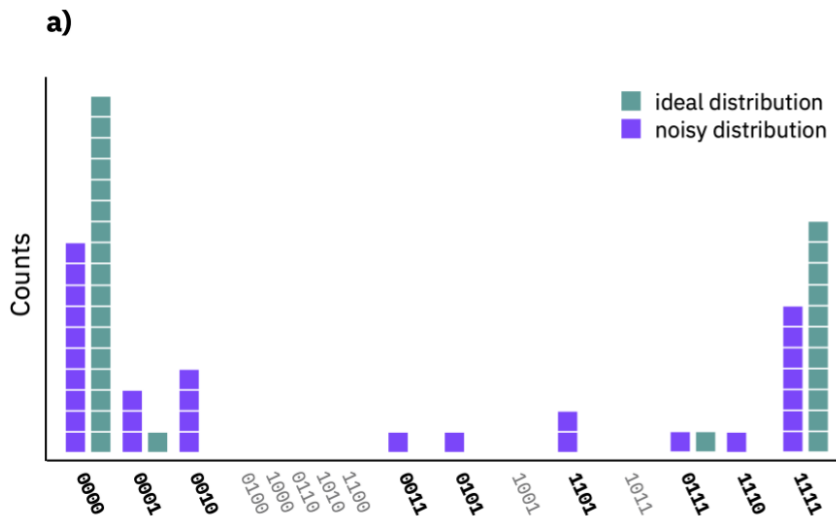
PEC for Estimator:

- can be enabled by setting 'pec_mitigation' to True in the Qiskit Runtime resilience options for Estimator
- Other options:
 - max_overhead: to set a limit on the sampling overhead (limiting the sampling overhead can cause the precision of the result to exceed the requested precision) (default value: 100)

```
estimator = Estimator(mode=backend)
estimator.options.resilience.pec_mitigation = True
estimator.options.resilience.pec.max_overhead = 100
```

Error mitigation for Sampler (mthree):

- a package for scalable quantum measurement error mitigation that need not explicitly form the assignment matrix, or its inverse
 - matrix-free measurement mitigation (M3) routine
- works in a **reduced subspace** defined by the **noisy input bitstrings that are to be corrected**
- number of unique bitstrings $\ll$ dimensionality of the full multi-qubit Hilbert space,
 - the resulting linear system of equations is nominally much easier to solveoften this linear equation is trivial to solve using LU decomposition, using only modest computing resources
- if the number of unique bitstrings is large, and / or one has very tight memory constraints, then the problem can be solved in a matrix-free manner using a preconditioned iterative linear solution method, e.g. the Generalized minimal residual (GMRES) or biconjugate gradient stabilized (BiCGSTAB) methods



- M3 is suitable for problems amenable to using quasi-probabilities
 - such as those formulated in terms of expectation values, or sampling problems where, for example, one is interested in the bit-string with the highest probability.
- Quasi-probabilities can be projected onto the nearest probability distribution if true probabilities are desired, but this makes error analysis more difficult.
- M3 works for mid-circuit measurements as well, provided that one is interested in ensemble averages, as opposed to correcting single-shot measurements; it cannot mitigate single-shot measurements used for conditional-gate logic.

Resilience level (for Estimator):

- The resilience level **specifies how much resilience to build against errors.**
- **Higher levels** generate **more accurate results**, at the **expense of longer processing times.**
- can be used to configure the cost/accuracy trade-off when applying error mitigation to your primitive query
- Error mitigation reduces errors (bias) in results by processing the outputs from a collection, or ensemble, of related circuits
- The degree of error reduction depends on the method applied
- The resilience level abstracts the detailed choice of error mitigation method to allow users to reason about the cost/accuracy trade that is appropriate to their application
- each level corresponds to a method or methods with increasing level of quantum sampling overhead to enable you experiment with different time-accuracy tradeoffs
- Estimator supports the following resilience levels. Sampler does not support resilience levels

Resilience Level	Definition	Technique
0	No mitigation	None
1 [Default]	Minimal mitigation costs: Mitigate error associated with readout errors	Twirled Readout Error eXtinction (TREX) measurement twirling
2	Medium mitigation costs. Typically reduces bias in estimators, but is not guaranteed to be zero-bias.	Level 1 + Zero Noise Extrapolation (ZNE) and gate twirling

Example code:

The following code uses Zero Noise Extrapolation and readout error mitigation by simply setting `resilience_level` 2.

```
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit_ibm_runtime import EstimatorV2 as Estimator

service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)

# Setting options during primitive initialization
estimator = Estimator(backend, options={"resilience_level": 2})
```

Custom error settings:

Estimator:

```
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit_ibm_runtime import EstimatorV2 as Estimator

service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)

estimator = Estimator(backend)
options = estimator.options
# Turn on gate twirling.
options.twirling.enable_gates = True
# Turn on measurement error mitigation.
options.resilience.measure_mitigation = True

print(f">>> gate twirling is turned on:
{estimator.options.twirling.enable_gates}")
print(f">>> measurement error mitigation is turned on:
{estimator.options.resilience.measure_mitigation}")
```


Custom error settings:

Sampler:

```
from qiskit_ibm_runtime import SamplerV2 as Sampler

# Estimator and Sampler have different options
sampler = Sampler(backend)
options = sampler.options

options.dynamical_decoupling.enable = True
# Turn on gate twirling. Requires qiskit_ibm_runtime 0.23.0 or
# later.
options.twirling.enable_gates = True

print(f">>> dynamical decoupling is turned on:
{sampler.options.dynamical_decoupling.enable}")
print(f">>> gate twirling is turned on:
{sampler.options.twirling.enable_gates}")
```

Turn off all error mitigation and error suppression:

Estimator:

```
from qiskit_ibm_runtime import EstimatorV2 as Estimator,
QiskitRuntimeService

# Define the service. This allows you to access IBM QPU.
service = QiskitRuntimeService()

# Get a backend
backend = service.least_busy(operational=True, simulator=False)

# Define Estimator
estimator = Estimator(backend)

options = estimator.options

# Turn off all error mitigation and suppression
options.resilience_level = 0
```

Qiskit Addons:

- a collection of modular tools for building utility-scale workloads powered by Qiskit
- These modular software components can plug into a workflow to scale or design new quantum algorithms
- Many of these addons are powered by the qiskit-addon-utils package

☐ Shaded lightcones

☐ Propagated Noise Absorption (PNA)

Shaded lightcones:

- the shaded lightcones (SLC) add-on uses Pauli propagation to reduce the number of error terms in a noise model that need to be mitigated.
- has the effect of reducing the sampling overhead for PEC workflows
- Reference: Eddins, Andrew, Minh C. Tran, and Patrick Rall. "Lightcone shading for classically accelerated quantum error mitigation." *arXiv preprint arXiv:2409.04401* (2024).

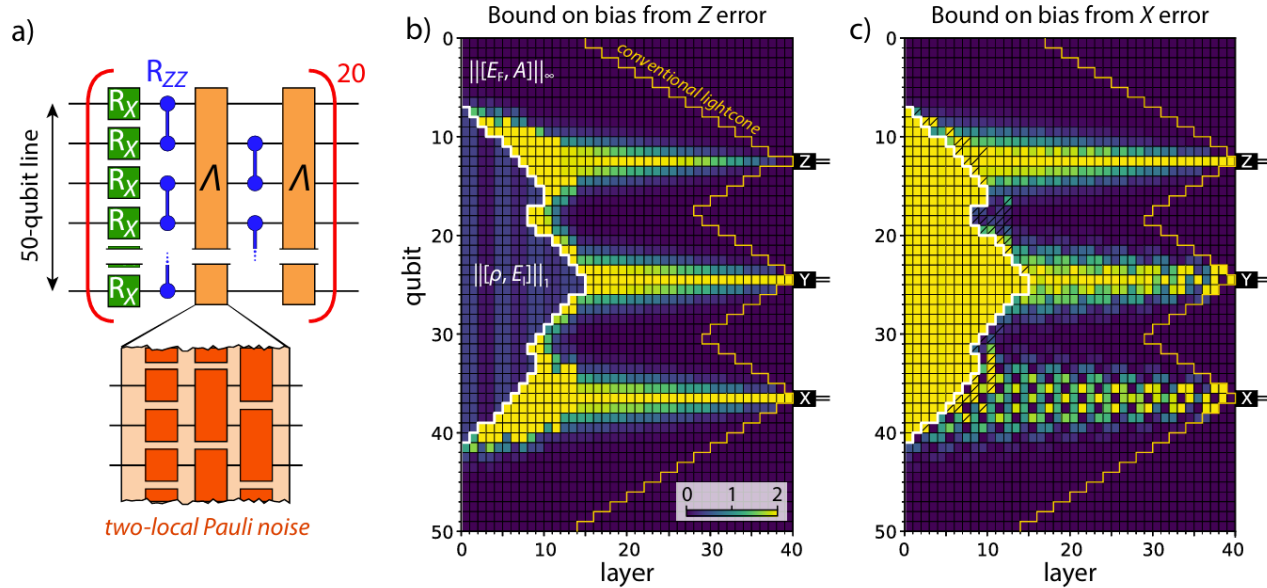
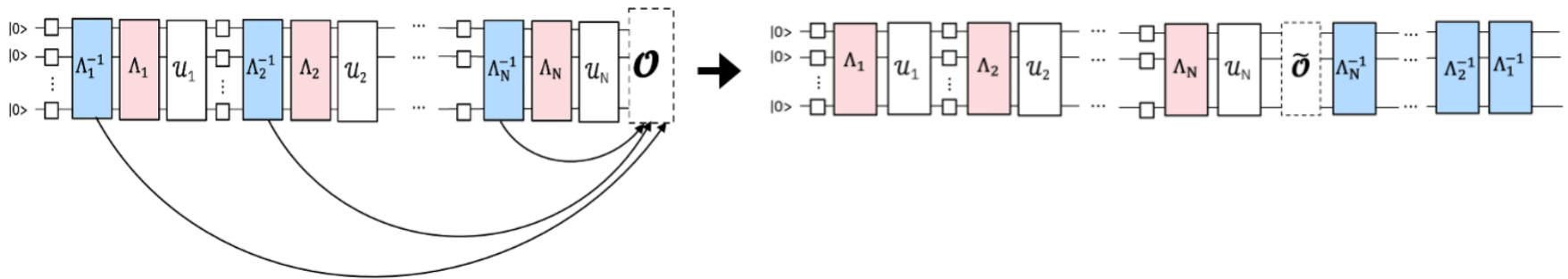


FIG. 3. Shaded lightcones for the noisy one-dimensional transverse-field Ising model Trotter circuit in (a), with 50 qubits, 40 layers of two-qubit R_{ZZ} gates, gate angles $\theta_X = \pi/16$ and $\theta_{ZZ} = -\pi/2$, and observable $A = X_{12}Y_{24}Z_{36}$. Each pixel bounds the bias contribution of an individual (b) Z or (c) X error somewhere in the circuit. Dark pixels indicate errors that would inflict little or no bias if neglected in PEC. A conventional lightcone based on commutation checks would fill all points left of the yellow boundary with the trivial bound of 2 (Fig. 1(a)). The white boundary separates bounds on the effect of the error on the initial state or on the observable, corresponding to the time-ordering partition T in Eq. (9). Slashes indicate commutator weights exceeding $N_{\max}$, where a looser triangle-inequality bound was computed instead of $||[E_F, A_F]||_{\infty}$. The complete shaded lightcone includes 12 such plots (Appendix F), collectively bounding all possible two-local Pauli errors. Moderate thresholds $B_{\max} = 5 \cdot 10^5$, $N_{\max} = 20$ were chosen for this pedagogical figure to enable computation on a laptop. This set of computations completed in ~ 10 hours on a laptop; trivial parallelization over some of the 23,640 error channels in the circuit could reduce this time (by a smaller factor), at a cost of more memory.

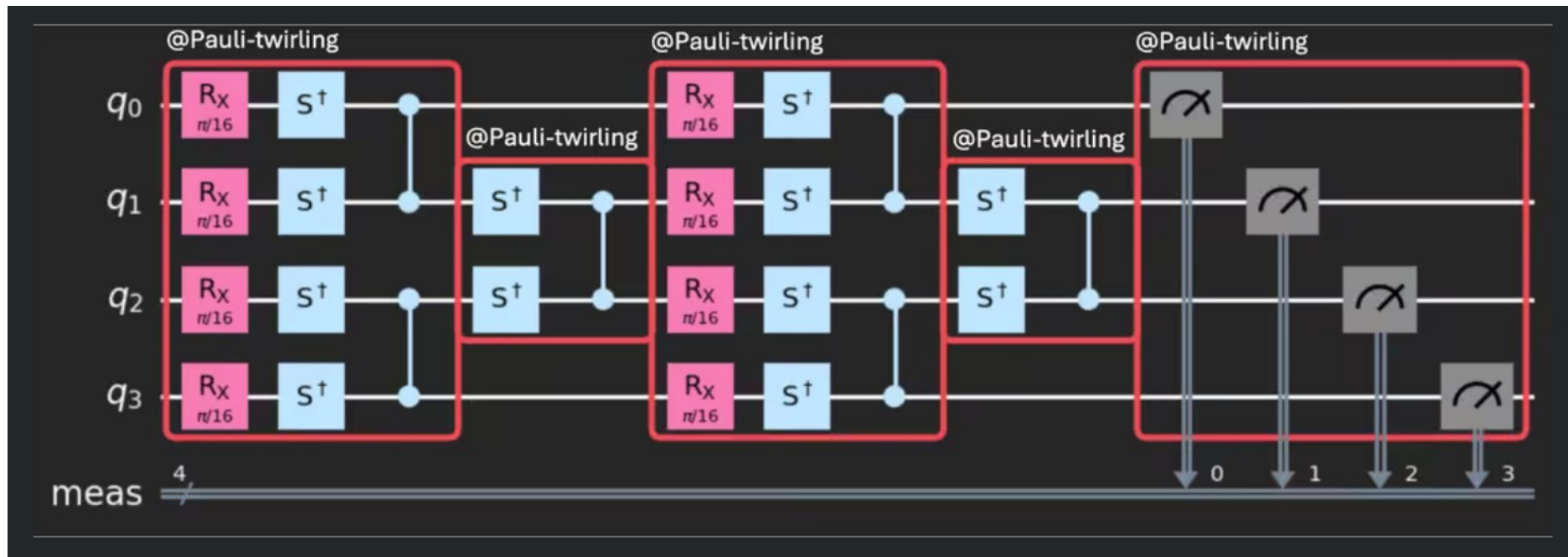
Propagated Noise Absorption (PNA):

- a technique for mitigating errors in observable expectation values by “absorbing” the inverses of the learned noise channels into the observable using Pauli propagation
- Each Pauli noise generator in the noise model is classically propagated to the end of the circuit and applied to the observable, resulting in a new observable that when measured on a QPU, mitigates the learned gate noise
- Reference: <https://qiskit.github.io/qiskit-addon-pna/>



Directed execution model (beta):

- Samplomatic
- Box construct
- simplex
- For example, one can stratify your circuit into boxes, add a twirling annotation to each box, and specify which twirling group to use, as shown in the following figure:



- Executor primitive
- NoiseLearnerV3
- Reference: <https://quantum.cloud.ibm.com/docs/en/guides/directed-execution-model>

References:

- <https://quantum.cloud.ibm.com/docs/en/guides/error-mitigation-overview>
- <https://quantum.cloud.ibm.com/docs/en/guides/error-mitigation-and-suppression-techniques>
- <https://quantum.cloud.ibm.com/docs/en/guides/addons>
- <https://quantum.cloud.ibm.com/docs/en/guides/algorithmiq-tem>
- <https://quantum.cloud.ibm.com/docs/en/tutorials>